

Demystifying Large Language Models (LLMs)

Understanding
Transformers,
Attention, and Beyond

Overview

Why Transformers?

Tokenization

Embeddings

Entire Flow

Inside Transformer

Attention & MLP

Training vs Inference

Insights for KV cache

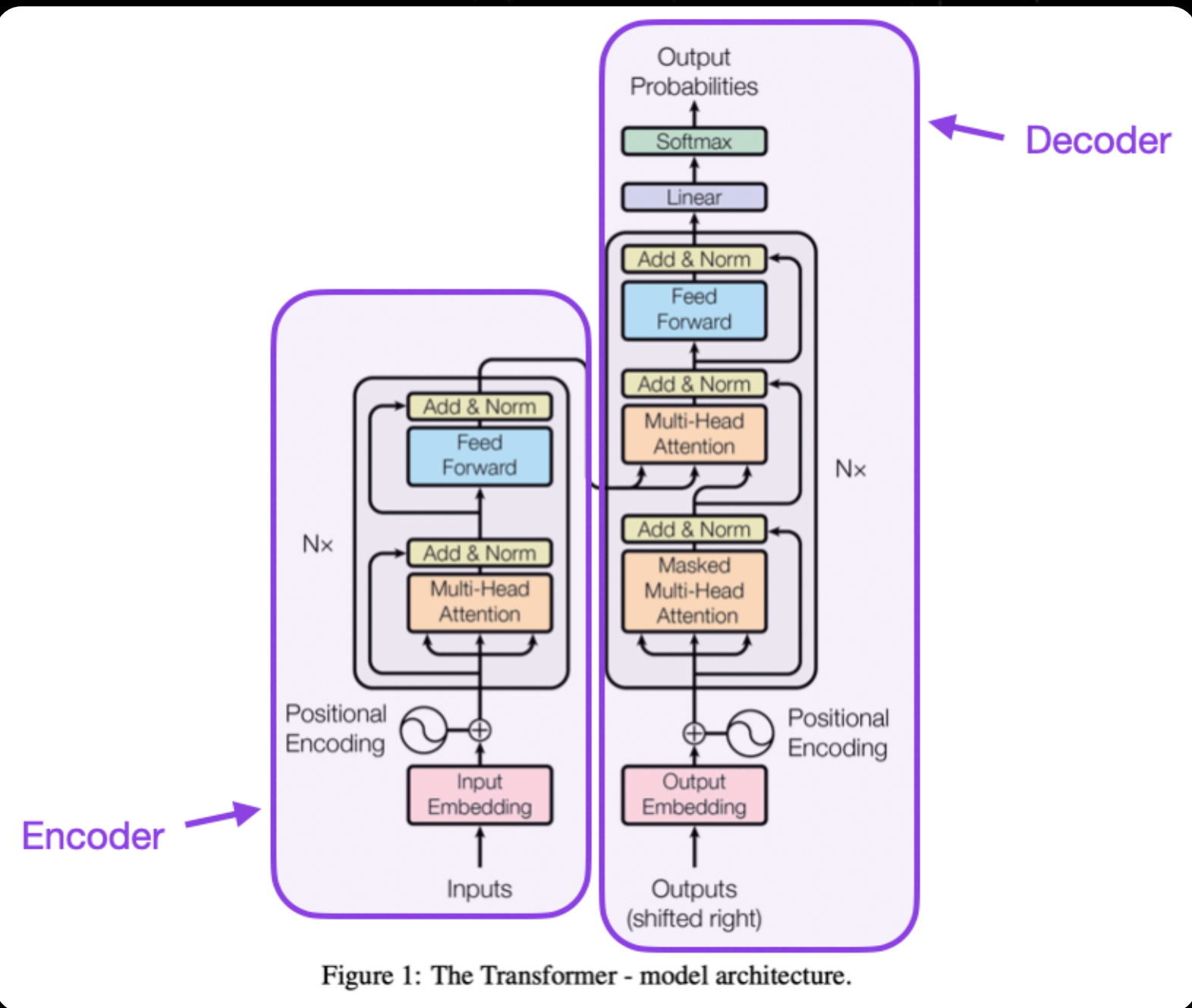


Figure 1: The Transformer - model architecture.

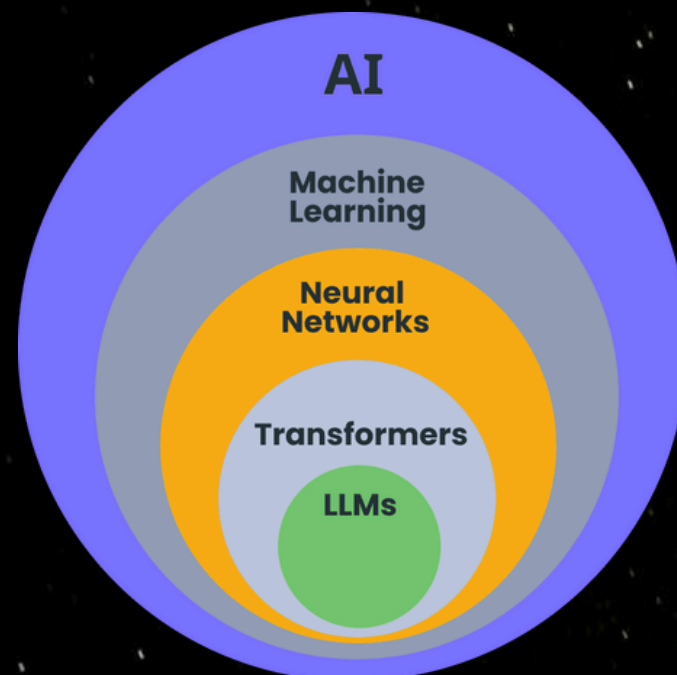
Why Transformers? The Revolution That Changed AI

RNN/LSTM Limitations

- Sequential processing bottleneck
- Vanishing gradient problems
- Poor parallelization capabilities
- Limited long-range dependency capture

Transformer Advantages

- Parallel processing architecture
- Superior long-range dependencies
- Highly scalable design
- Efficient training on modern hardware

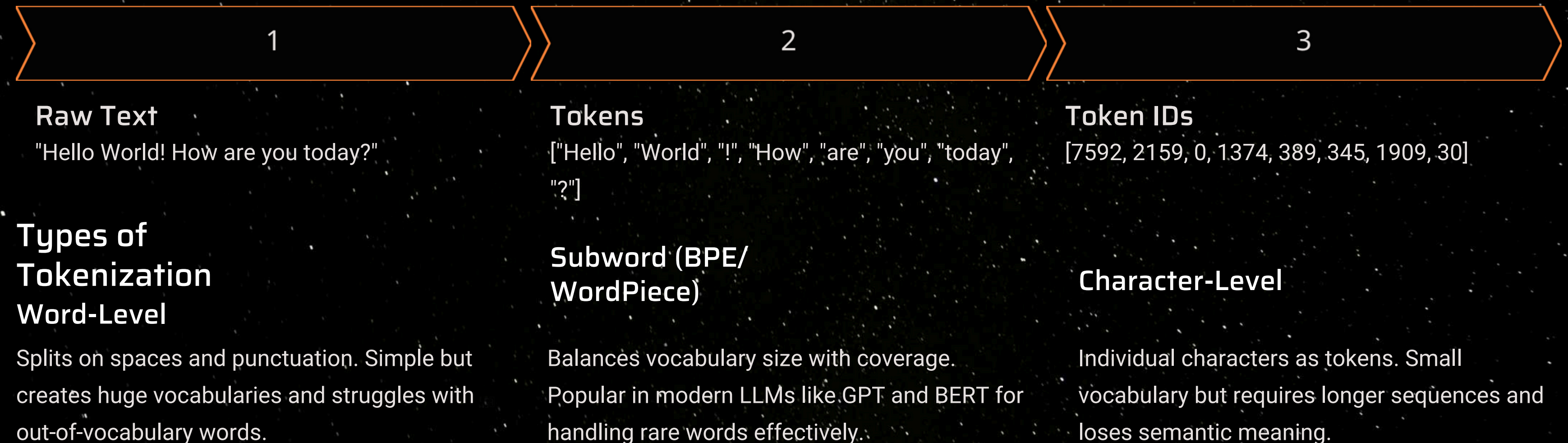


Before Transformers emerged in 2017, natural language processing relied heavily on Recurrent Neural Networks (RNNs) and Long Short-Term Memory (LSTM) networks. These architectures processed text sequentially, word by word, creating significant computational bottlenecks and struggling with long-distance relationships in text.

Transformers revolutionized the field by introducing the attention mechanism, enabling models to process all tokens simultaneously while maintaining awareness of relationships across the entire sequence. This breakthrough made training faster, more efficient, and dramatically improved performance on complex language tasks.

Tokenization: Breaking Down Language

Tokenization is the critical first step where human language meets machine processing. It's how we transform continuous text into discrete units that neural networks can understand and manipulate.



Tiktokenizer

gpt-4o

System

You are a helpful assistant



User

Content



Add message

```
<|im_start|>system<|im_sep|>You are a helpful assistant<|im_end|><|im_start|>user<|im_sep|><|im_end|><|im_start|>assistant<|im_sep|>
```

Token count
16

```
<|im_start|>system<|im_sep|>You are a helpful assistant<|im_end|><|im_start|>user<|im_sep|><|im_end|><|im_start|>assistant<|im_sep|>
```

```
200264, 17360, 200266, 3575, 553, 261, 10297, 2918  
6, 200265, 200264, 1428, 200266, 200265, 200264, 17  
3781, 200266
```

☐ Show whitespace

Built by dqbd. Created with the generous help from Diagram.



Tokenization

- Each word is broken into a token(subwords) given a TOKEN ID, for the sequence “Hello world, How are you?”



- In reality, it is more like:

Hello World,How are you?

13225, 5922, 11, 5299, 553, 481, 30

Now we have numbers [7592, 1917, 0], but they're just arbitrary IDs.

Token 7592 could be 'Hello' or 'Goodbye' - the number itself means nothing.

How do we give these numbers meaning?

"The cat sat on the mat because it was comfortable."

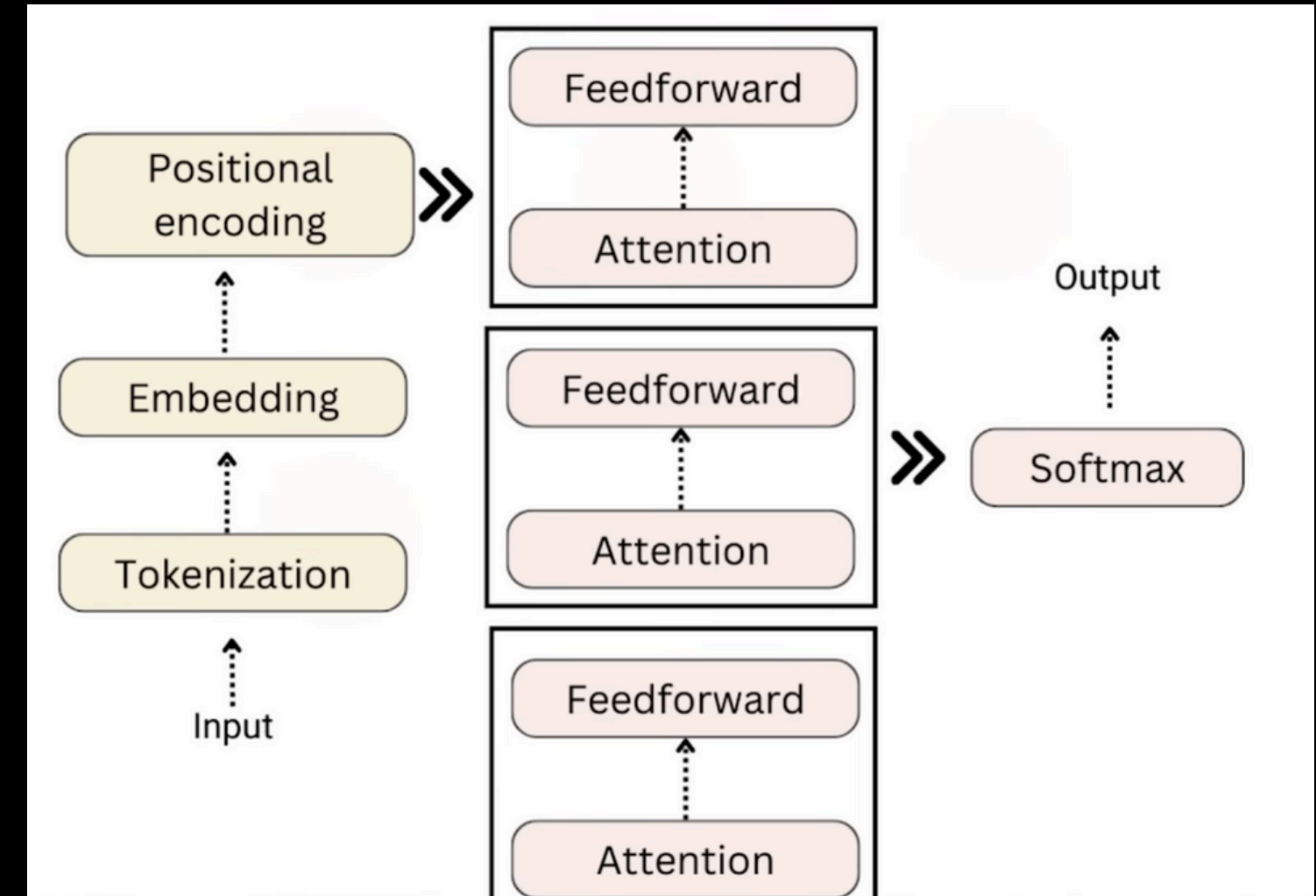
Problem 1: Words Have Multiple Meanings

- "it" could refer to the cat OR the mat
- "comfortable" could describe the cat's feeling OR the mat's texture
- A human instantly knows "it" refers to the cat, but how?

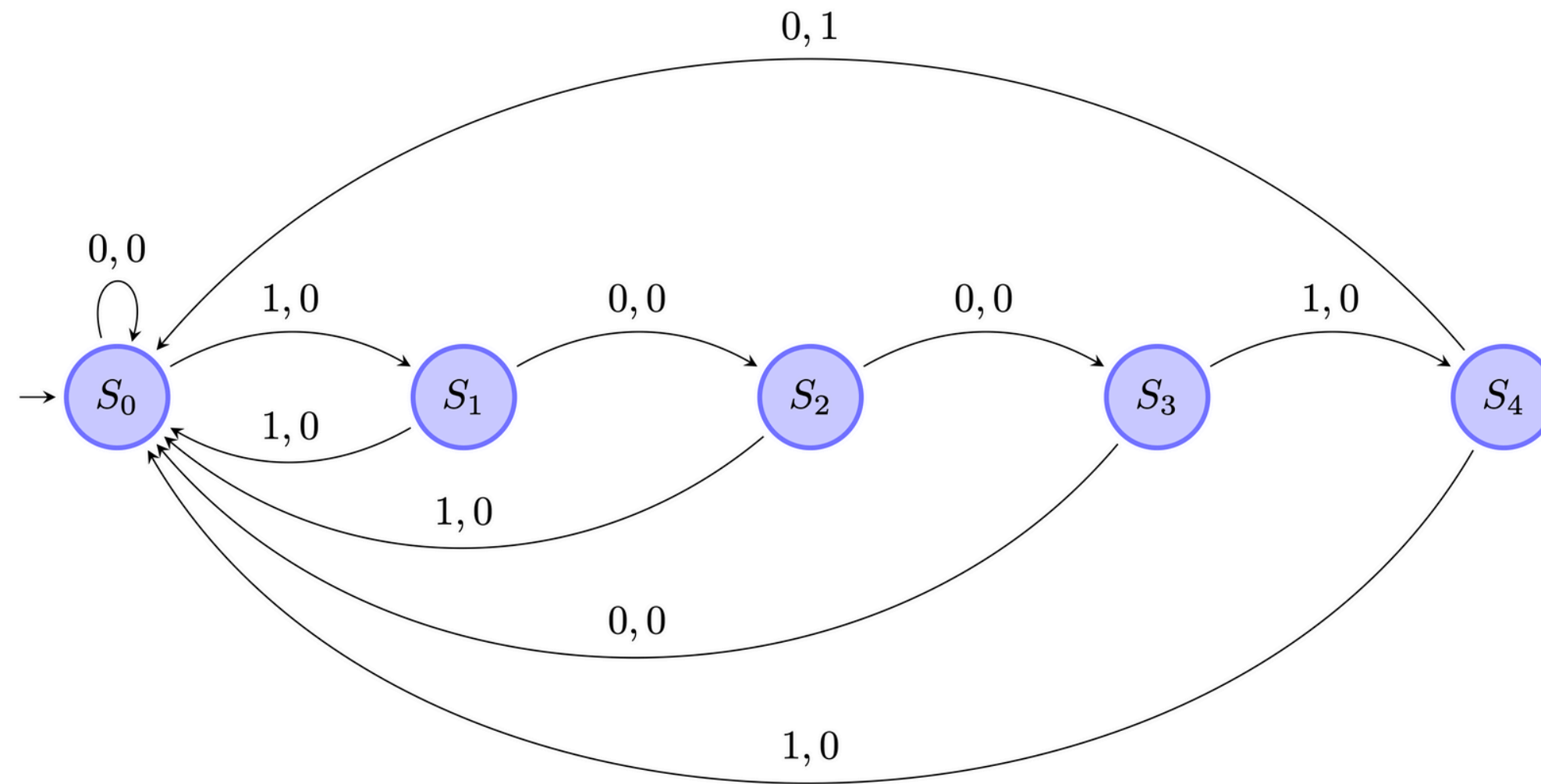
Problem 2 : Context matters in long distances

Problem 3: Order Matters, But So Does Parallel Processing

- "Cat sat on mat" vs "Mat sat on cat" = completely different meanings
- But ideally, we want to process all words simultaneously for speed
- How do we maintain order while going fast?



Finite State Machine



Transformers themselves aren't finite state machines they're much more powerful.
But FSMs are useful around transformers for tokenization, pattern checking, and
constraining outputs, especially when you want to enforce strict rules during generation

Embeddings - Translation Layer

What you see: "The cat sat on the mat"
What the computer sees: ???

Computers only understand numbers. They can't directly process the word "cat" - they need numbers like [0.2, -0.4, 0.7, 0.1, ...]

Idea: If you wanted to teach math to someone who only spoke in colors, you'd need a translation system. Embeddings are our translation system from human language to "computer language"

When we give a model a sentence like "The cat sat on the mat," the model isn't actually "seeing" cats or mats the way humans do. Computers only work with numbers. So instead of words, the model uses vectors (lists of numbers) to represent each word these are called word embeddings.

At the start, these embeddings are just random numbers. During training, as the model processes lots of text, it uses backpropagation to adjust the numbers so that words appearing in similar contexts (like "cat" and "dog") end up with vectors that are close together. After training, the model has "learned" useful representations of words not by knowing what a cat physically is, but by knowing how the word "cat" is used in language.

Every word gets converted into a list of numbers (called a vector). But not just any random numbers - meaningful numbers.

Example: Let's say we have a 3-dimensional embedding system (real ones use a lot more dimensions):

"cat" → [0.2, -0.4, 0.7]

"dog" → [0.1, -0.3, 0.8]

"apple" → [0.9, 0.2, -0.5]

"orange" → [0.8, 0.3, -0.4]

The Position Problem

Consider these two sentences:

- "The cat sat on the mat"
- "The mat sat on the cat"

With just word embeddings, both sentences would have identical representations because they contain the exact same words! But they mean completely different things.

The problem: Transformers process all words simultaneously ("lights on" analogy), so they don't naturally know the order.

Imagine the sentence "The cat sat on the mat."

The word embeddings might be random vectors like [0.2, 0.6, ...]. We add positional vectors:

Position 1 → [0.1, 0.3, ...]

Position 2 → [0.4, 0.5, ...]

Position 3 → [0.7, 0.2, ...]

Position 4 → [0.9, 0.8, ...]

This makes "cat" at position 2 different from "cat" at position 5.

The good insight here is that words with similar meanings should have similar numbers.

Imagine a 3D space where each word is a point:

- All animals cluster together in one area
- All fruits cluster together in another area
- All colors cluster together somewhere else

This isn't programmed manually - the AI learns these relationships by reading millions of sentences and noticing patterns like:

- "The cat meowed" and "The dog barked" (similar contexts)
- "I ate an apple" and "I ate an orange" (similar usage)

1. banana
2. orange
3. yellow
4. finch
5. monarch

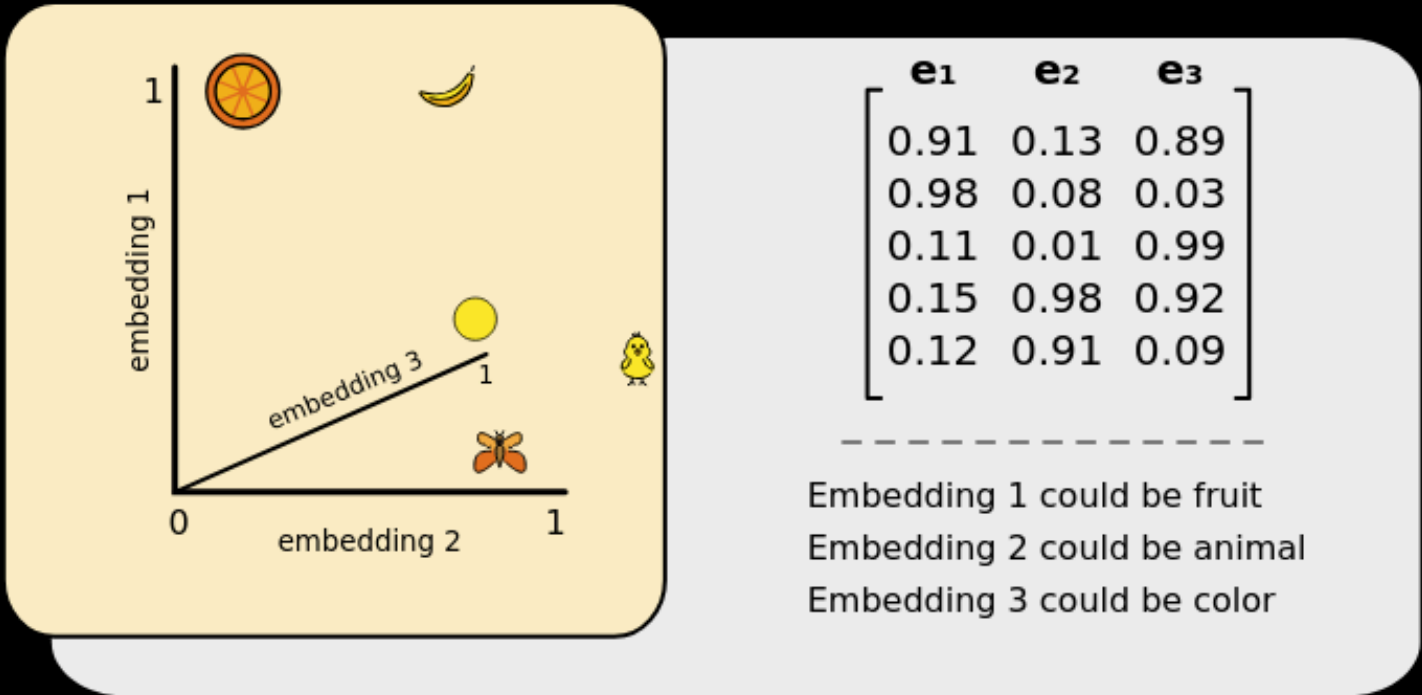
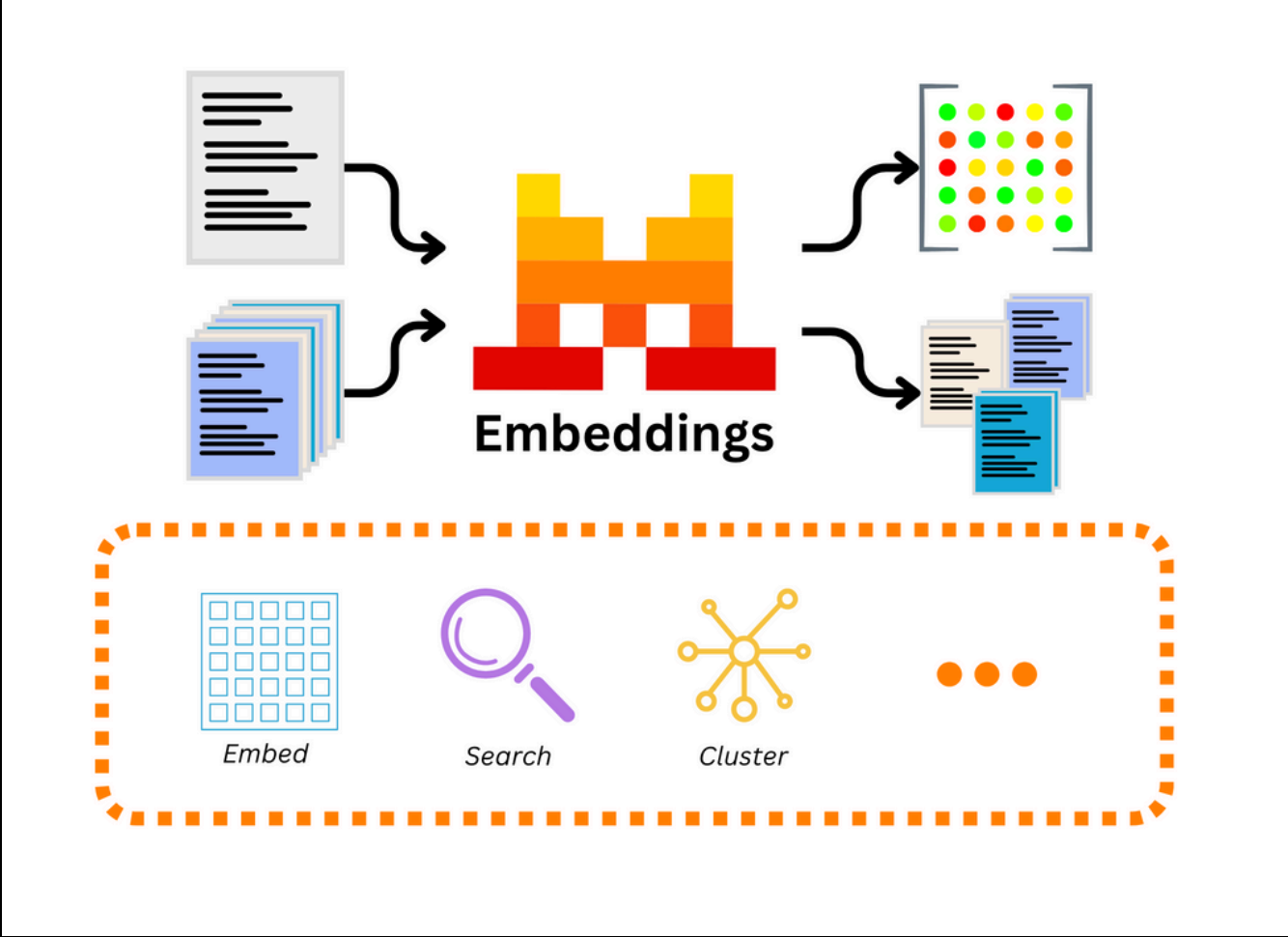
The Famous Word2Vec Example

King - Man + Woman ≈ Queen

In vector math:

[King vector] - [Man vector] + [Woman vector] ≈ [Queen vector]

This means the AI automatically learned that the relationship between "King" and "Queen" is the same as between "Man" and "Woman"!



The Complete LLM Pipeline

Understanding how text flows through a Large Language Model requires seeing the complete picture. Each stage transforms the input in specific ways, building toward the final output generation.

1

Input Text

Raw human language enters the system

2

Tokenization

Text is broken into discrete tokens using BPE or similar algorithms

3

Token Embeddings

Tokens are converted to high-dimensional vectors capturing semantic meaning

4

Positional Encoding

Position information is added since Transformers don't inherently understand sequence order

5

Transformer Layers

Multiple layers of attention and feedforward networks process the embeddings

6

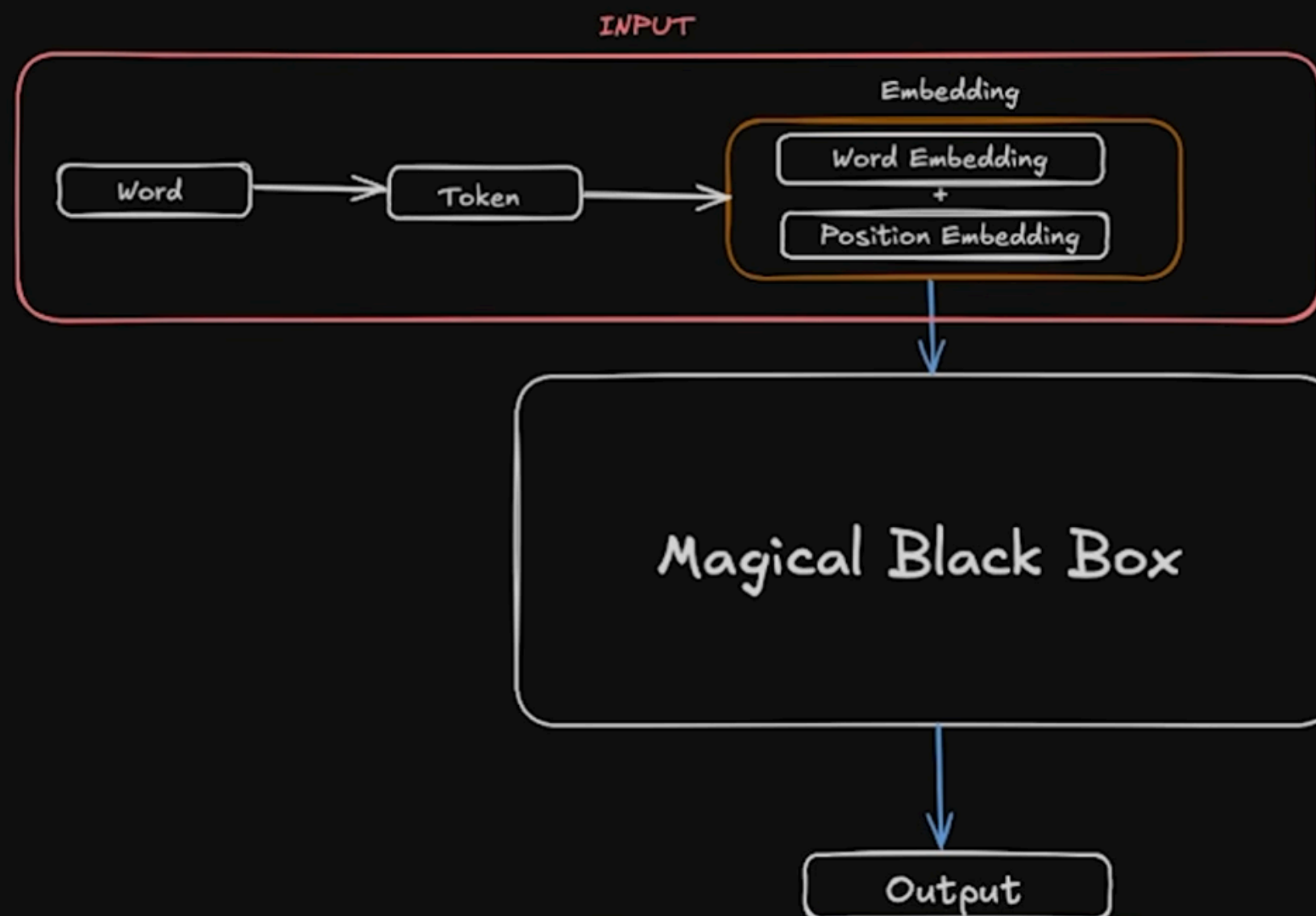
Output Generation

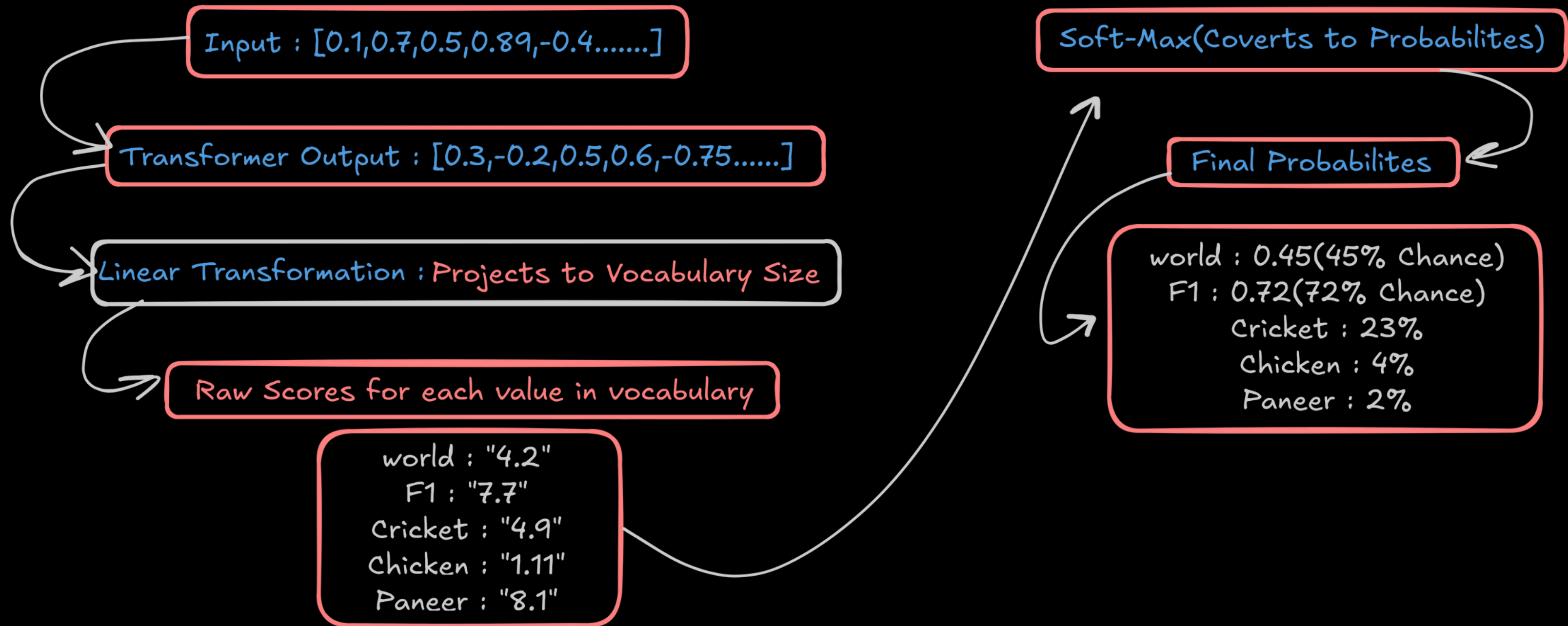
Final layer produces probability distributions over the vocabulary for next token prediction

This pipeline represents the fundamental architecture that powers everything from chatbots to code generation. Each component plays a crucial role in transforming human language into mathematical representations that can be processed and manipulated by neural networks.

The Flow

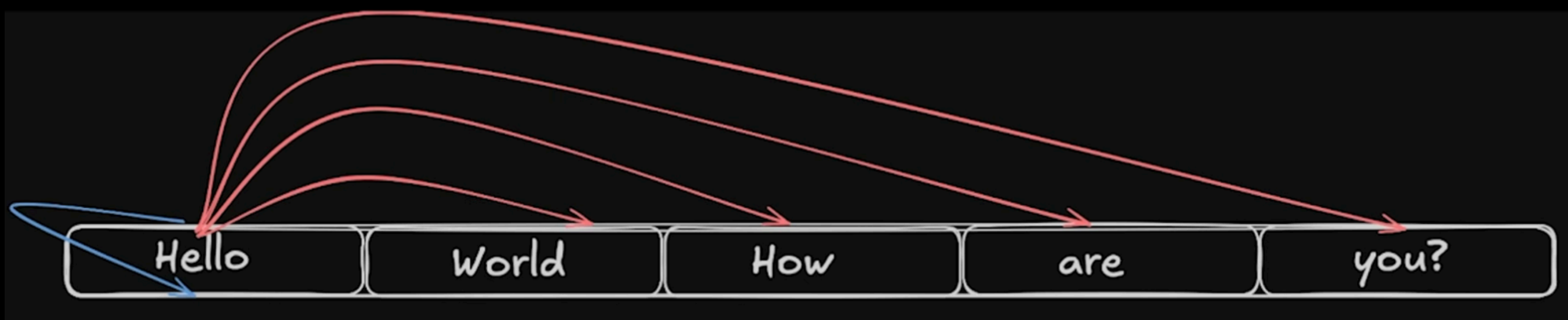
- Sequence of Words -> Sequence of Word Embedding -> Add positional embeddings -> Magical BlackBox -> Next Prediction





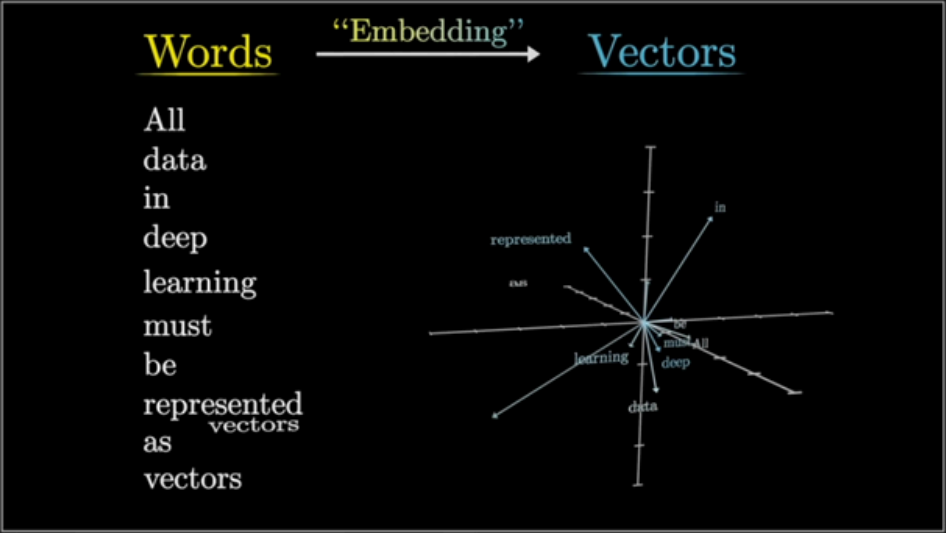
Attention!!!!

How exactly does Attention work? How do words "talk to each other"? What are these mysterious "Keys," "Values," and "Queries"?



Inside Transformer

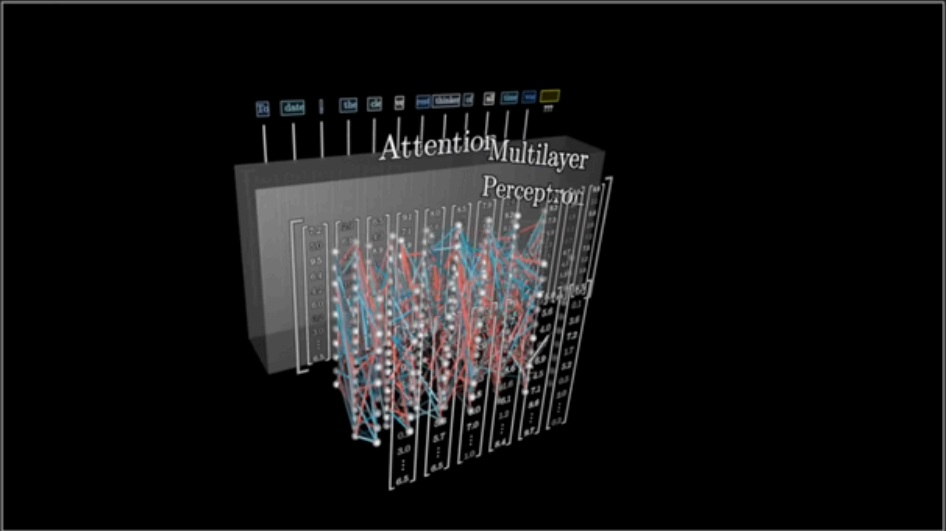
Embedding



Attention



MLPs



Unembedding



Attention Mechanism: The Heart of Understanding

Self-attention is the breakthrough that makes Transformers so effective. It allows every token to directly interact with every other token in the sequence, creating rich contextual representations.

Queries (Q)

What information is this token looking for? Each token generates a query vector representing what it wants to attend to.



Keys (K)

What information does this token contain? Key vectors represent the content that can be attended to by other tokens.

Values (V)

The actual information to be retrieved. When attention weights are high, more of this token's value is included in the output.

Multi-Layer Perceptron (MLP)

After attention, each token's representation passes through a feedforward network that applies non-linear transformations. This MLP typically has a hidden layer 4x larger than the model dimension, using ReLU or GELU activation functions. The MLP allows the model to perform complex reasoning operations on the attention-processed information.

The combination of attention and MLP creates a powerful processing unit that can capture both relational patterns (attention) and perform complex transformations (MLP) on the input data.

Inside the Transformer: Core Components

The Transformer's power comes from its elegant architecture combining two key mechanisms: attention for capturing relationships between tokens, and feedforward networks for complex transformations.



Multi-Head Attention

Enables the model to focus on different aspects of the input simultaneously. Each "head" learns different types of relationships - some might focus on syntax, others on semantics.



Feedforward Network

A simple but powerful MLP that applies non-linear transformations to the attention output. Typically much larger than the attention layer, containing most of the model's parameters.



Residual Connections

Skip connections that add the input directly to the output, enabling training of very deep networks by preventing vanishing gradients.

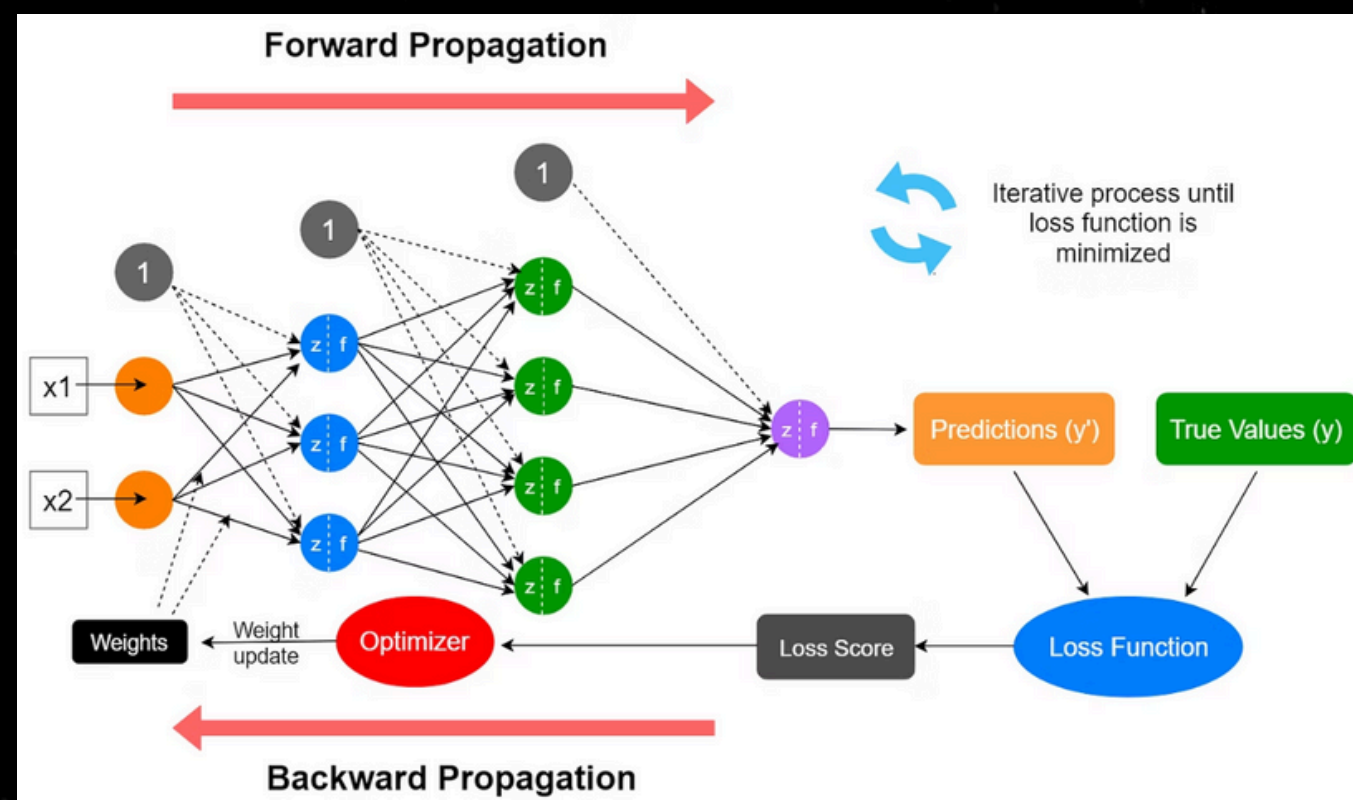


Layer Normalization

Normalizes inputs to each sublayer, stabilizing training and improving convergence speed in deep networks.

These components work together in each transformer layer, with modern LLMs stacking dozens or even hundreds of these layers to create increasingly sophisticated representations of language.

Training vs Inference: Two Different Worlds



- **Teacher Forcing:** Model sees entire sequence, predicts next token at each position
- **Parallel Processing:** All predictions computed simultaneously
- **Backpropagation:** Gradients flow backward to update weights
- **Massive Scale:** Trained on trillions of tokens from diverse text sources

175B

Parameters
GPT-3 model size

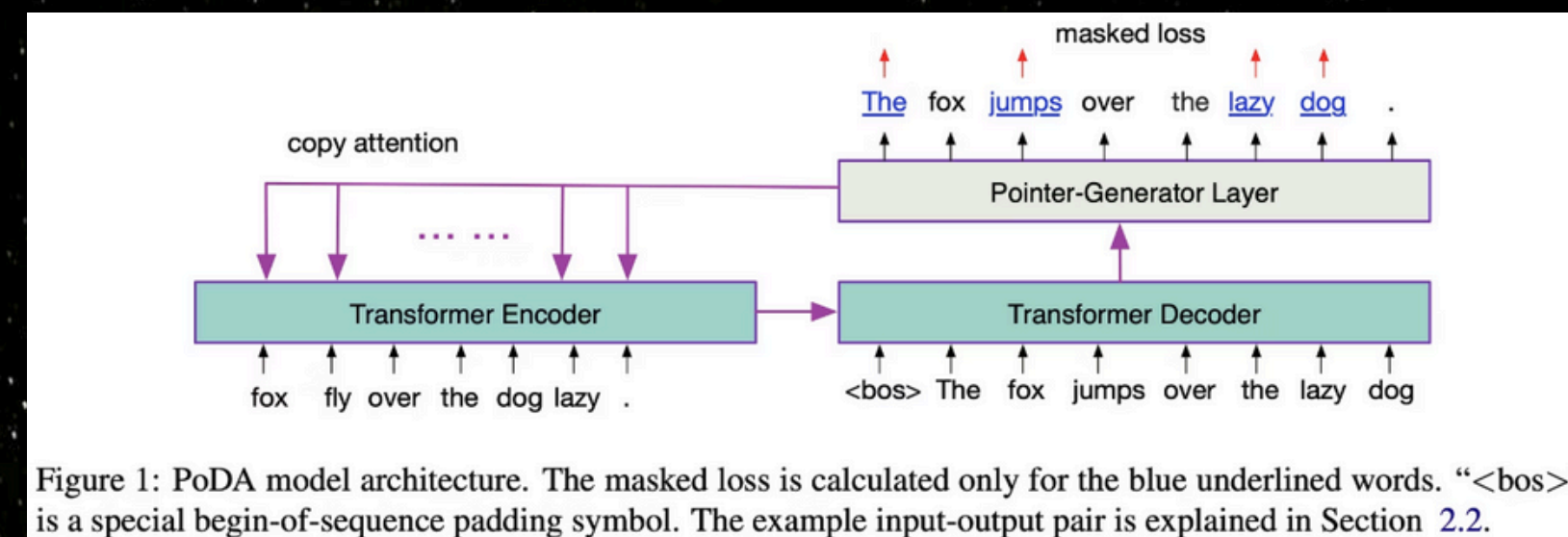
300B

Training Tokens
Approximate training data size

1000x

Compute Ratio
Training vs inference computational cost

This fundamental difference between training and inference shapes how we optimize and deploy LLMs. Training requires massive parallel computation, while inference demands efficient sequential generation with careful memory management.



- **Autoregressive:** Generates one token at a time, sequentially
- **No Teacher Forcing:** Uses its own predictions as input for next token
- **Forward Only:** No gradient computation needed
- **Efficiency Focus:** Optimizations like KV caching critical for speed

Scale Effects: When Bigger Becomes Better

One of the most remarkable discoveries in modern AI is that language models exhibit emergent abilities as they scale up. Capabilities that seem impossible at smaller scales suddenly appear with sufficient model size and training data.



Basic Completion

Small models (1B parameters) can complete sentences and simple patterns but struggle with complex reasoning.



Few-Shot Learning

Medium models (10B+ parameters) start showing ability to learn from examples within the prompt context.



Complex Reasoning

Large models (100B+ parameters) demonstrate emergent abilities like chain-of-thought reasoning and multi-step problem solving.



Expert-Level Performance

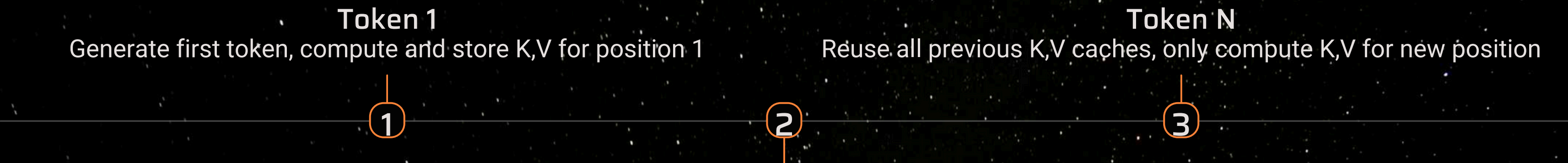
Massive models (500B+ parameters) approach or exceed human performance on many specialized tasks.

"The scaling laws suggest that model performance improves predictably with compute, data, and parameters - but emergent abilities appear unpredictably at certain scale thresholds."

Understanding these scaling effects is crucial for predicting the capabilities of future models and planning computational resources. The relationship between scale and capability continues to drive the race toward ever-larger language models.

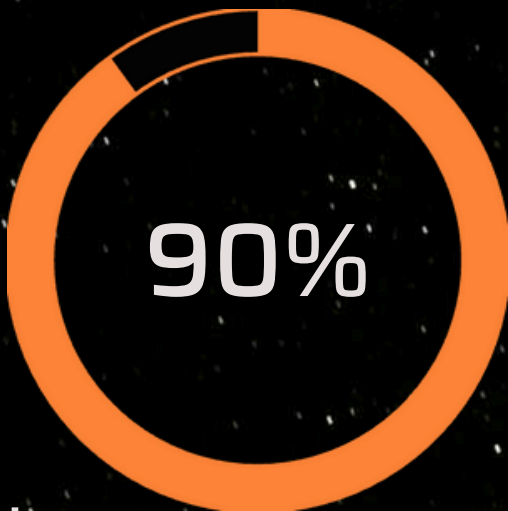
KV Cache: Optimizing Inference Speed

Key-Value caching represents one of the most important optimizations in LLM inference. Without it, generating long sequences would be computationally prohibitive due to redundant calculations.



Without KV Cache

Each new token requires recomputing attention for all previous tokens. Time complexity grows quadratically with sequence length, making long conversations extremely slow.

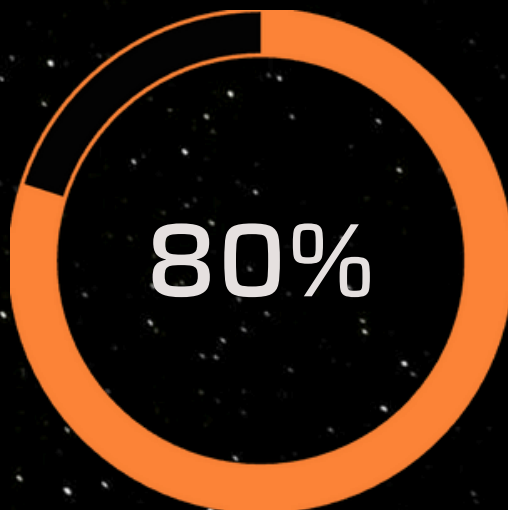


Speed Improvement

Typical inference acceleration with KV caching

With KV Cache

Store previously computed key-value pairs in memory. New tokens only require computing attention with cached values, reducing time complexity to linear growth.



Memory Usage

Additional GPU memory required for cache storage

KV caching is essential for practical LLM deployment, enabling real-time conversations and long-form content generation. However, it requires careful memory management as cache size grows with sequence length.

Let's End it
Schroedinger's Cat
is gonna Sleep now